Microsoft Teams → Google Chat & Meet Migration & Coexistence

Implementation Solution Guide

Version 1.0.0 · 2026-07-06

Contents

Executive Summary — Microsoft Teams → Google Chat & Meet Migration & Coexistence 5

Honest about what migrates . . . 5

What you get . . . 5

What this delivers . . . 6

The deployment model . . . 6

Outcome . . . 6

1. Solution Overview — Microsoft Teams → Google Chat & Meet Migration & Coexistence 6

What this solution is . . . 6

The Teams → Chat/Meet model . . . 6

Honest scope — what a Teams → Chat migration can and can't carry . . . 7

Meetings → Google Meet . . . 8

Coexistence during the cutover . . . 8

The named artifacts . . . 9

Workstream sequencing . . . 9

What this is not . . . 9

The deployment model . . . 10

2. Requirements Guide — Microsoft Teams → Google Chat & Meet Migration & Coexistence 10

2.1 Administrative access to the source Microsoft Teams estate . . . 10

2.2 Google Workspace with Chat and Meet enabled, and licensing . . . 10

2.3 Identity & provisioning into Google . . . 10

2.4 A channel → space mapping . . . 11

2.5 A meetings-transition design (Teams meetings → Meet + interop) . . . 11

2.6 A migration tooling decision (native or third-party), with limits acknowledged . . . 11

2.7 Legal-hold / retention handling for the Teams source . . . 12

2.8 A wave plan . . . 12

2.9 Per-wave cutover runbook and go/no-go gate . . . 12

2.10 An adoption & change-management plan . . . 12

Checklist . . . 13

Global Requirements — Google Workspace & Gemini Solutions 13

1. Google Workspace / Cloud Identity tenant . . . 13

2. Source Microsoft 365 / Entra environment (for migration & coexistence) . . . 14

3. DNS control for the routed domains . . . 14

4. Identity & provisioning source of truth . . . 14

5. A pilot scope (OU / group) and change control . . . 14

6. Security, compliance & governance stakeholders . . . 15

3. Implementation Guide — Microsoft Teams → Google Chat & Meet Migration & Coexistence 15
Step 1 — Assess: baseline the Teams estate and stand up governance and adoption 15
Step 2 — Configure: map channels to spaces and decide the migration path 16
Step 3 — Design and stand up meeting coexistence 17
Step 4 — Build the wave plan 17
Step 5 — Author the per-wave cutover runbook and go/no-go gate 17
Step 6 — Run Wave 0 (the pilot) end to end 18
Step 7 — Validate the pilot and fix the runbook, not just the wave 18
Step 8 — Expand wave by wave 18
Step 9 — Hypercare and decommission 19
5. Testing Guide — Microsoft Teams → Google Chat & Meet Migration & Coexistence 19
Test 1 — Baseline and wave assignment are complete 19
Test 2 — Readiness: identity, licensing, and Chat/Meet enabled 19
Test 3 — Spaces created with correct membership 19
Test 4 — History scope imported as designed (and limits honored) 20
Test 5 — Files reachable from the space 20
Test 6 — Meetings work on Google Meet 20
Test 7 — Coexistence: cross-platform meetings interop 20
Test 8 — Rollback is proven available 20
Test 9 — Adoption and coverage reported 20
Pass/fail gate 21
6. Troubleshooting Guide — Microsoft Teams → Google Chat & Meet Migration & Coexistence
A. Users expected migrated chat history that isn't there 21
B. A space is stuck in import mode or was deleted 21
C. The import fails partway or won't take all teams 22
D. A space has the wrong or missing members 22
E. Cross-platform meetings fail during coexistence 22
F. A team is split across Chat and Teams mid-conversation 23
G. A wave fails its go/no-go gate 23
H. The same problem recurs every wave 23
I. A cutover needs to be reversed 24
J. Graph scripts fail to connect 24
K. Diagnostics quick reference 24
L. When to escalate 24
7. Best Practices — Microsoft Teams → Google Chat & Meet Migration & Coexistence 24
8. Deliverables Summary — Microsoft Teams → Google Chat & Meet Migration & Coexistence

Documentation 26
Named artifacts 26
PowerShell scripts 27
The Migration playbook (outline) 27
What the buyer receives 28
Outcome 28
Appendix A — PowerShell Scripts 29

Executive Summary — Microsoft Teams → Google Chat & Meet Migration & Coexistence

Teams isn't just chat — it's **channels, meetings, and shared files**, and your users are mid-conversation when you move them. This solution moves collaboration from **Microsoft Teams to Google Chat and Google Meet** as a governed migration: a mapping of Teams **channels to Chat spaces**, a transition of **meetings to Meet**, a plan for the shared **files**, a **coexistence** design so a Teams user and a Chat user can still work together during the cutover, and **adoption** guidance so people actually land on the new tools — executed **pilot-first and rollback-ready**.

Most Teams migrations stumble the same way: they treat it as a chat-only move and forget meetings and files; they overpromise a perfect history copy the tools can't deliver and lose user trust; coexistence is an afterthought, so cross-platform meetings break while half the estate is on each platform; and nothing is proven first, so every wave re-learns the same lessons. This solution removes those failure modes with a named mapping, honest scope, meeting interop, and a gated, wave-based cutover.

Honest about what migrates

This solution is deliberately straight about the limits, because overpromising is how these projects fail. Google's native path migrates **channel conversations into spaces**, but **1:1/private chats are not carried**, some message formats and Teams-specific features don't survive, history imports into hidden **import-mode** spaces on a **90-day** clock, and the migration is a **productivity feature, not a compliance archive** — so the Teams source is retained for legal hold ([Google for Developers — import data to Google Chat](https://developers.google.com/workspace/chat/import-data), [Google Workspace Help — run a chat migration](https://knowledge.workspace.google.com/admin/migrate/run-a-chat-migration), [Google Workspace Help — set up a chat migration](https://knowledge.workspace.google.com/admin/migrate/set-up-a-chat-migration)). The effort goes into **going-forward collaboration** — the right spaces, meetings on Meet, and adoption — not a museum of old messages.

What you get

- A **Solution Overview** of the Teams → Chat/Meet model — channels→spaces, meetings→Meet, files→Drive,

and coexistence — with the honest message-history scope and the named migration artifacts.

- A **Requirements Guide** with verify / implement-if-missing for every prerequisite: source Teams

access, a Google tenant with Chat/Meet enabled and licensed, identity/provisioning, the channel→space mapping, the meetings-transition/interop design, the tooling decision with its limits, legal-hold handling, the wave plan, the runbook/gate, and adoption.

- An **Implementation Guide** of precise, ordered steps: baseline the Teams estate, map channels to

spaces, decide the migration path, stand up meeting coexistence, run a named pilot wave, validate, and expand — with hypercare and a decommission that retains the source for legal hold.

- **Five PowerShell scripts** — Teams estate discovery, per-team migration/coexistence readiness

validation, a coverage/adoption report, safe remediation, and a wave target-group builder — read-only against Microsoft Graph except the two write scripts, which default to `-WhatIf`.

- **Testing and Troubleshooting** guides with explicit per-wave gates (spaces created, meetings work, users active), plus Best Practices for adoption and coexistence.

What this delivers

Collaboration moved from Teams to Chat and Meet with the honest scope set and met: channels mapped to spaces, meetings transitioned to Meet, users kept working through coexistence, every cutover gated by an explicit go/no-go, and the full runbook proven on a pilot wave before any production business unit moves.

The deployment model

Assess, then Configure, then Pilot, then Validate, then Expand — with a per-wave go/no-go gate before every subsequent wave, meeting interop kept live through the split, and the Teams source retained for legal hold until the estate is fully migrated.

Outcome

People land on Chat and Meet without being stranded — teams stay together through the cutover, cross-platform meetings keep working, spaces and meetings are actually used, and collaboration continues without the "where did my conversation go?" surprise that sinks careless migrations.

1. Solution Overview — Microsoft Teams → Google Chat & Meet Migration & Coexistence

What this solution is

Teams isn't just chat. It is **channels, meetings, and shared files**, and users are mid-conversation when you move them. This solution moves collaboration from **Microsoft Teams to Google Chat and Google Meet** as a governed migration: a mapping of Teams **channels to Google Chat spaces**, a transition of **meetings to Google Meet**, a plan for the **shared files** that live inside Teams, a **coexistence** design so a Teams user and a Chat user can still work together during the cutover, and **adoption** guidance so people actually land on the new tools. It runs pilot-first and rollback-ready, using the category deployment model: Assess, then Configure, then Pilot, then Validate, then Expand.

It is **tool-agnostic**. Google offers a native channel-message migration and there are third-party tools; this solution governs *what maps to what, in what order, and how you prove it* — the specific console steps of any one migration tool are deferred to that tool's own docs. Where the included scripts read the source, they read **Microsoft Teams / Microsoft 365** through Microsoft Graph, read-only.

The Teams → Chat/Meet model

Teams bundles several capabilities behind one app. The migration treats each as its own workstream with its own target and its own honest scope.

Teams object	Google target	How it moves
A **team** and its **standard channels**	A set of **Google Chat spaces** (one space per channel)	Channel *conversations* can be migrated into spaces; membership is re-created from the team/channel roster
Channel messages / conversation history	**Messages inside a space**	Migrated via an import path (native or third-party) with real limits — see "Honest scope" below
1:1 chats and group (private) messages	Google Chat DMs / group DMs	**Not carried by the native channel migration** — treated as a fresh start unless a third-party tool covers them; message history limits apply
Teams meetings (scheduled + recurring)	**Google Meet**	Meeting *links* are re-created in Google Calendar/Meet; existing invites are re-issued or interop-bridged during coexistence
Files inside channels (the SharePoint/OneDrive behind Teams)	**Google Drive / shared drives**	Moved by the Drive migration workstream (deferred to the SharePoint/OneDrive → Google Drive solution); this solution maps *which* space points at *which* Drive location
The people	Google Chat + Meet users	Provisioned into Google first (identity workstream); adoption drives real usage

Honest scope — what a Teams → Chat migration can and can't carry

Message-history migration has real, documented limits. Overpromising here is how these projects lose trust, so this solution states the limits up front and plans around them:

- **Channels map to spaces; conversations can be migrated, but not everything in them.** Google's native path migrates **channel conversations from Teams into Chat spaces** ([Google Workspace Updates — migrate Teams messages to Chat](https://workspaceupdates.googleblog.com/2024/12/open-beta-migrate-microsoft-teams-messages-to-google-chat.html), [Google Workspace Updates — migrate with CloudM](https://workspaceupdates.googleblog.com/2025/02/migrate-data-from-microsoft-teams-to-google-chat.html)).
- **Some message content does not survive.** Rich-text formatting such as underlining, information in **message cards**, **nested lists**, and Teams-specific features like **Streams, praise, and approvals** are **not** migrated by the native tool ([Google Workspace Help — run a chat migration](https://knowledge.workspace.google.com/admin/migrate/run-a-chat-migration)).
- **1:1 and private/group chats are out of scope for the channel migration.** The native path targets

named spaces; **direct messages and unnamed group messages are not supported** by import mode ([Google for Developers — import data to Google Chat](https://developers.google.com/workspace/chat/import-data)).

- **Import happens into hidden "import-mode" spaces on a 90-day clock.** Data is imported into spaces

in **import mode** that end users can't see; the import (including finalizing/rolling out the spaces) must complete within **90 days** or the space is deleted and unrecoverable ([Google for Developers — import data to Google Chat](https://developers.google.com/workspace/chat/import-data)). Once a space is **finalized**, you can't add more historical messages to it.

- **Scale is bounded per import.** The native tool imports up to **1,000 teams per import**; more teams

means additional imports ([Google Workspace Help — run a chat migration](https://knowledge.workspace.google.com/admin/migrate/run-a-chat-migration)).

- **This is a productivity move, not a compliance archive.** Google is explicit that the chat migration

is a productivity feature and **not intended for legal/retention purposes** — keep the Teams source (or an eDiscovery export) for anything under legal hold ([Google Workspace Help — set up a chat migration](https://knowledge.workspace.google.com/admin/migrate/set-up-a-chat-migration)).

The practical consequence: **plan for a "history-lite" cutover.** Migrate active channel conversations where it adds value, set the expectation that not every message/format carries, keep the Teams source read-only for reference and legal hold, and put the design effort into **going-forward collaboration** (the right spaces, the right members, meetings on Meet, and adoption) rather than a perfect history copy.

Meetings → Google Meet

Teams meetings become **Google Meet** meetings. Two things make this safe during the cutover:

- **Re-issue meeting links.** New and recurring meetings are created as Google Meet events on Google

Calendar for migrated users; existing recurring Teams invites are re-issued as Meet invites for the wave.

- **Interop during coexistence.** While the estate is split, cross-platform meetings still work:

calendar sync keeps events/RSVPs consistent across Google and Teams, and Meet↔Teams interop lets each side join the other's meetings so no one is stranded on the wrong client ([Microsoft Learn — calendar syncing between Google Workspace and Teams](https://learn.microsoft.com/en-us/microsoftteams/setup-calendar-syncing-with-google-workspace), [Google Workspace Updates — Meet–Teams interoperability](https://workspaceupdates.googleblog.com/2026/02/meet-hardware-microsoft-teams-intero.html)). Interop is transitional — it keeps the split estate working, not a permanent end state.

Coexistence during the cutover

Because users move in waves, a migrated user and a not-yet-migrated user must still collaborate. The coexistence design defines: which users are on Chat/Meet vs Teams at any point, how meetings interop across the two (calendar sync + Meet↔Teams join), and the expectation that **live chat does not bridge 1:1 across the two platforms** — so waves are kept to coherent teams/business units to minimize cross-platform chat during the split. Coexistence is **time-boxed** and decommissioned as waves complete.

The named artifacts

Author these once and reuse them for every wave. Fixed names keep the migration legible across owners and waves.

Artifact	Name
Source Teams estate baseline	`GWCHAT Discovery - Teams Estate - Baseline`
Channel → space mapping	`GWCHAT Mapping - Channels to Spaces - Program`
Meetings transition design	`GWCHAT Meetings - Teams to Meet - Transition`
Coexistence design	`GWCHAT Coexistence - Chat & Meet - Transition`
Wave plan	`GWCHAT Wave Plan - Segmentation - Teams`
Per-wave cutover runbook	`GWCHAT Runbook - Cutover - Per Wave`
Go/no-go gate	`GWCHAT Gate - Go/No-Go - Per Wave`
Adoption & comms plan	`GWCHAT Adoption - Change & Comms - Program`

Workstream sequencing

Ordered by dependency, not convenience:

• **Identity first.** Users and groups must exist in Google (via directory sync — GCDS or SCIM) before any space, meeting, or file lands, and before coexistence can address them.

• **Files and identity underpin spaces.** A channel's files move via the Drive workstream; the channel→space mapping records which Drive location each space points at, so a space isn't created without its content home.

• **Coexistence before the first wave.** Meeting interop (calendar sync + Meet↔Teams) must be live before anyone cuts over, so a migrated and a not-yet-migrated user can still meet.

• **Chat, meetings, files per wave.** Within a wave, spaces are created and (optionally) history imported, meetings are re-issued on Meet, and the wave's Drive content is finalized.

• **Decommission last.** Interop and the Teams source footprint are retired only after the final wave clears hypercare — with the Teams source retained for legal hold as long as policy requires.

What this is not

This solution governs the mapping, sequencing, coexistence, and adoption — the orchestration. It does **not** re-document the internal console steps of any specific migration tool, and it does **not** move the underlying files (that's the Drive migration solution). Its scripts read **Microsoft Teams / Microsoft 365** through Microsoft Graph, read-only; they move no messages, create no meetings, and change nothing on

either platform.

The deployment model

> *Baseline the Teams estate and design the mapping once, stand up meeting coexistence, prove the full cutover on a named pilot wave, validate against the go/no-go gate, then expand wave by wave until the estate is on Chat and Meet — the repeatable way: Assess, then Configure, then Pilot, then Validate, then Expand.*

2. Requirements Guide — Microsoft Teams → Google Chat & Meet Migration & Coexistence

Verify every requirement before running the migration. Each entry gives **Description**, **Why**, **Verify**, and **Implement-if-missing**. Category-wide prerequisites (the Google Workspace tenant, the source Microsoft 365 / Entra environment, DNS control, the identity source of truth, a pilot scope, and governance stakeholders) are in `global-requirements-google.md`; this guide adds what the Teams → Chat/Meet migration specifically needs.

2.1 Administrative access to the source Microsoft Teams estate

Description: Administrative read access to the source **Microsoft Teams / Microsoft 365** tenant through Microsoft Graph — enough to enumerate teams, their standard channels, channel membership, and the meetings/calendars in scope — captured as `GWCHAT Discovery - Teams Estate - Baseline`. Why: The channel→space mapping, the wave plan, and the meetings-transition design are all built from an accurate picture of the Teams estate; without it, spaces are mapped and waves are sized by guesswork. Verify: You can enumerate teams and channels for a representative sample; run `GWCHAT-Discovery.ps1`. Implement-if-missing: Obtain the read roles from the Microsoft 365/Teams tenant owner and run `GWCHAT-Discovery.ps1`, then add the details the API cannot read (per-channel file volumes, meeting counts, integrations) by hand and declare it the baseline of record.

2.2 Google Workspace with Chat and Meet enabled, and licensing

Description: A Google Workspace tenant where **Google Chat and Google Meet are enabled** for the in-scope users, with the edition/licensing the migration path needs (the native Teams-message migration requires target users to have **active licenses and Chat enabled**). Why: Spaces cannot be created for, and messages cannot be imported to, users who are not licensed and Chat-enabled; meetings cannot be re-issued on Meet for users without Meet. Verify: In the Google Admin console, confirm Chat and Meet are ON for the pilot OU and the pilot users are licensed; run `GWCHAT-Validation.ps1` for the wave. Implement-if-missing: Enable Chat and Meet for the target OUs and assign licenses before scheduling the pilot wave.

2.3 Identity & provisioning into Google

Description: Users and groups flow from the identity source of truth into Google **before** their spaces or meetings are created — via Google Cloud Directory Sync (GCDS) or SCIM-based provisioning — per the source of truth defined in the global requirements. Why: A space's membership and a meeting's invitees

resolve to Google accounts; nothing can be created or addressed until the accounts exist, and coexistence cannot route to them. Verify: A sample wave's users and groups appear in the Google Admin console before any space is created; run `GWCHAT-Validation.ps1` for that wave. Implement-if-missing: Stand up GCDS/SCIM provisioning against the source of truth and confirm a test population provisions before scheduling Wave 0.

2.4 A channel → space mapping

Description: A decision record — `GWCHAT Mapping - Channels to Spaces - Program` — that maps each in-scope **team/standard channel** to a **Google Chat space** (with the space name, member roster, and the Drive location that holds the channel's files), and records the **history scope** decided for each space (conversations imported, or fresh start). Why: Waves must create spaces the *same* way; deciding naming, membership, and history scope per channel at cutover time guarantees inconsistent spaces and rework. The mapping is also where the honest message-history limits are written down so expectations are set once. Verify: Every in-scope channel maps to exactly one space with a name, a member source, a Drive target, and a history decision; the mapping is agreed by the migration owners. Implement-if-missing: Build the mapping in Implementation Step 2 from the discovery baseline; freeze it before Wave 0.

2.5 A meetings-transition design (Teams meetings → Meet + interop)

Description: A design — `GWCHAT Meetings - Teams to Meet - Transition` — for how scheduled/recurring **Teams meetings become Google Meet** meetings (re-issue links for migrated users) and how **meeting interop** works while the estate is split (calendar sync between Google and Teams, and Meet↔Teams join). Why: During the months the estate is split, a migrated user and a not-yet-migrated user must still meet; without a design, recurring meetings break at cutover and cross-platform meetings fail. Verify: The design names the re-issue approach for recurring meetings and the interop mechanism, and a test cross-platform meeting joins successfully in a lab ([Microsoft Learn — calendar syncing with Google Workspace](https://learn.microsoft.com/en-us/microsoftteams/setup-calendar-syncing-with-google-workspace), [Google Workspace Updates — Meet–Teams interoperability](https://workspaceupdates.googleblog.com/2026/02/meet-hardware-microsoft-teams-intero.html)). Implement-if-missing: Design the meetings transition in Step 3, stand up interop on a test population, and prove it before the first production wave. Interop is transitional — set its intended end.

2.6 A migration tooling decision (native or third-party), with limits acknowledged

Description: A recorded decision on the **message-migration path** — Google's native channel-message migration, a third-party tool, or a deliberate **fresh-start** (no history) — with the documented limits acknowledged in writing: no 1:1/private-chat migration by the native path, unsupported message formats/features, the 90-day import-mode window, the 1,000-teams-per-import cap, and that the migration is **not a compliance archive** ([Google for Developers — import data to Google Chat](https://developers.google.com/workspace/chat/import-data), [Google Workspace Help — run a chat migration](https://knowledge.workspace.google.com/admin/migrate/run-a-chat-migration), [Google Workspace Help — set up a chat migration](https://knowledge.workspace.google.com/admin/migrate/set-up-a-chat-migration)). Why: The path drives the runbook and the user comms; deciding it late, or promising a perfect history copy the tools

can't deliver, is the classic way these projects lose trust. Verify: The tooling decision names the path and lists the accepted limits; the tool's own console steps are referenced (not re-authored here). Implement-if-missing: Choose the path in Step 2 and record the accepted limits in the mapping artifact before Wave 0.

2.7 Legal-hold / retention handling for the Teams source

Description: A decision on what happens to the **Teams source** after migration — kept read-only for reference and any legal hold, or exported via eDiscovery — because the chat migration is a productivity feature, **not** a retention/compliance archive. Why: If the source is retired assuming history migrated, data under legal hold can be lost; the migrated copy is not a defensible archive. Verify: The retention owner has signed off on how the Teams source is preserved for as long as policy requires. Implement-if-missing: Engage the legal/eDiscovery stakeholder from the global requirements and record the source-retention decision before any Teams decommission.

2.8 A wave plan

Description: A segmentation of the in-scope population into sized, bounded, scheduled waves — named `GWCHAT Wave Plan - Segmentation - Teams` — with Wave 0 as the pilot, kept to **coherent teams/business units** so cross-platform chat during the split is minimized. Why: Waves turn the estate into executable, gated units; keeping a team together in one wave avoids a team being split across Chat and Teams mid-conversation (which live chat does not bridge 1:1). Verify: Every in-scope user is in exactly one wave, waves keep teams intact where possible, and the pilot wave is identified. Implement-if-missing: Build the wave plan from the discovery baseline; build each wave's target group with `GWCHAT-Set-ScopeTargeting.ps1`.

2.9 Per-wave cutover runbook and go/no-go gate

Description: A repeatable per-wave runbook — `GWCHAT Runbook - Cutover - Per Wave` — with the fixed sequence (readiness → spaces & history → meetings/interop → validation → rollback) and an explicit `GWCHAT Gate - Go/No-Go - Per Wave` that must pass before the next wave starts. Why: Every wave executes the same governed cutover; an ungoverned cutover is declared done before validation, and a failed one with no rollback becomes an incident. Verify: The runbook exists with all five stages and a documented rollback, and the gate criteria are defined and owned. Implement-if-missing: Author the runbook and gate in Step 5, prove them on Wave 0, and reuse them unchanged for every subsequent wave.

2.10 An adoption & change-management plan

Description: A comms/training plan — `GWCHAT Adoption - Change & Comms - Program` — that prepares each wave's users for Chat and Meet (what moved, what didn't, where to find files, how meetings changed), set up alongside the technical plan, not bolted on at cutover. Why: A migration where spaces exist but no one uses them has failed; adoption is what turns a cutover into a working platform. Verify: The plan names the comms cadence and role-based training per wave and is part of each wave's readiness. Implement-if-missing: Build the adoption plan in Step 1 and make it a readiness gate for every wave.

Checklist

• Source Teams estate baselined — `GWCHAT-Discovery.ps1` → `GWCHAT Discovery - Teams Estate - Baseline`

• Google Chat + Meet enabled and licensed for in-scope users

• Identity/provisioning into Google live — `GWCHAT-Validation.ps1`

• Channel → space mapping frozen — `GWCHAT Mapping - Channels to Spaces - Program`

• Meetings-transition + interop design proven — `GWCHAT Meetings - Teams to Meet - Transition`

• Migration path + accepted limits recorded (native / third-party / fresh-start)

• Teams source retention / legal-hold decision signed off

• Wave plan built (teams kept intact, one user per wave) — `GWCHAT Wave Plan - Segmentation - Teams`

• Per-wave runbook + go/no-go gate authored — `GWCHAT Runbook - Cutover - Per Wave`

• Adoption & comms plan live — `GWCHAT Adoption - Change & Comms - Program`

Global Requirements — Google Workspace & Gemini Solutions

These requirements apply to **every** solution in the **Google** category. They are the common foundation each Google Workspace / Gemini solution assumes is already in place (or stands up as part of its own Requirements section). Verify each before starting a solution's implementation steps.

> ***Vendor boundary (applies to every Google solution).*** *The **Google Admin console** (admin.google.com), **Google Cloud / Cloud Identity**, and the **Microsoft / Entra** sides are authored at exact, current click-by-click detail from the vendors' public documentation. **Gemini Enterprise** capabilities that are in preview or rolling out are **cited or flagged "verify in your tenant"** — never invented. **Third-party migration tools** (e.g. for mail/Drive/Chat data movement) and **third-party enterprise apps** (ServiceNow, SAP, Databricks, etc.) are integrated at the pattern/architectural level; their internal console steps are deferred to the vendor's own docs.*

1. Google Workspace / Cloud Identity tenant

• **Description:** A provisioned Google Workspace (or Cloud Identity) tenant with a **super-admin** account for configuration, and the appropriate Workspace edition/licensing for the capability in scope (e.g. Enterprise Plus for Vault/CAA/DLP, Gemini/AI add-on where required).

• **Why required:** All configuration, policy, and API work is performed against this tenant.

• **How to verify:** Sign in to **admin.google.com** as a super-admin; confirm edition and license counts under **Billing / Subscriptions**.

• **How to implement if missing:** Provision the tenant and licenses through Google or an authorized reseller before starting.

2. Source Microsoft 365 / Entra environment (for migration & coexistence)

• **Description:** For migration/coexistence/identity solutions, administrative access to the source **Microsoft 365** tenant and **Microsoft Entra ID** (Global Administrator or the least-privilege roles the solution names).

• **Why required:** Migration, directory sync, SSO federation, and coexistence read from and change the Microsoft side.

• **How to verify:** Confirm access to the M365 admin center and the Entra admin center with the required role.

• **How to implement if missing:** Obtain the role from the M365/Entra tenant owner.

3. DNS control for the routed domains

• **Description:** Ability to create/modify **DNS records** (MX, SPF, DKIM, DMARC, TXT, CNAME) for the domains being migrated, coexisting, or verified.

• **Why required:** Domain verification, mail routing/coexistence, and authentication all depend on DNS changes.

• **How to verify:** Confirm access to the authoritative DNS zone for each domain.

• **How to implement if missing:** Coordinate with whoever controls DNS; stage changes under change control (mail-flow changes are high-impact).

4. Identity & provisioning source of truth

• **Description:** A defined **identity source of truth** (on-prem AD, Entra ID, or an HR/IGA system) and the sync mechanism (e.g. **Google Cloud Directory Sync (GCDS)** or SCIM-based provisioning).

• **Why required:** Consistent users/groups across ecosystems underpin SSO, licensing, and access.

• **How to verify:** Confirm which directory is authoritative and how objects flow into Google.

• **How to implement if missing:** Define the source of truth and provisioning model before configuring SSO or migrating.

5. A pilot scope (OU / group) and change control

• **Description:** A **pilot organizational unit or group** to validate on before fleet-wide rollout, and a **change-control** process for production changes (especially mail flow, DNS, identity, and DLP).

• **Why required:** Every solution follows an **Assess** → **Configure** → **Pilot** → **Validate** → **Expand** model;

the pilot proves the design and change control protects production.

• **How to verify:** Confirm a named pilot OU/group exists and a change window/approval path is available.

• **How to implement if missing:** Create a pilot OU/group and agree the change-control process with the

owners before making production changes.

6. Security, compliance & governance stakeholders

• **Description:** Engagement with the **security (CISO), legal/eDiscovery, and data-governance**

stakeholders whose policies the configuration must satisfy (DLP, retention, Vault/Purview, and any regulatory regime such as ITAR/DFARS/Assured Controls where applicable).

• **Why required:** Data governance, eDiscovery, DLP, and regulated-workload settings must align to

enterprise policy and legal requirements, not just technical defaults.

• **How to verify:** Confirm the applicable policies and named stakeholders/approvers.

• **How to implement if missing:** Identify the owners and obtain the policy/compliance requirements

before configuring governance controls.

*The deployment model across every Google solution is the same: **Assess** → **Configure** → **Pilot** → **Validate** → **Expand**, with explicit pass/fail gates and a documented rollback for production changes.*

3. Implementation Guide — Microsoft Teams → Google Chat & Meet Migration & Coexistence

Follow these steps in order. This guide runs the migration end to end: baseline the Teams estate, map channels to spaces and design the meetings transition, stand up meeting coexistence, prove the full cutover on a named pilot wave, then expand wave by wave until the estate is on Chat and Meet. Artifact names are fixed — create them with the exact names shown and reuse them for every wave. The internal console steps of any specific migration tool, and the movement of the underlying files, are deferred to that tool's docs and to the Drive migration solution; this guide governs the mapping, sequencing, coexistence, and adoption.

> *Model: baseline and map once, stand up meeting coexistence, prove the cutover on a pilot wave, validate against the gate, then expand. The repeatable way: Assess, then Configure, then Pilot, then Validate, then Expand.*

Step 1 — Assess: baseline the Teams estate and stand up governance and adoption

1. Run `GWCHAT-Discovery.ps1` to capture the source baseline: teams, standard channels per team, channel

membership counts, and the meeting/calendar signals visible through Microsoft Graph. Record it as `GWCHAT Discovery - Teams Estate - Baseline`, then add what the API cannot read (per-channel file volumes, recurring-meeting counts, third-party integrations) by hand.

1. Stand up the program governance: named Responsible/Accountable owners for each cutover stage, a

recurring go/no-go forum, and the change-control process from the global requirements.

1. Start the adoption plan `GWCHAT Adoption - Change & Comms - Program` now — comms and role-based

training are built alongside the technical plan, not added at cutover.

1. Record gaps against the Requirements checklist. Do not proceed past a missing baseline, missing

identity source of truth, or an unresolved legal-hold decision for the Teams source.

Step 2 — Configure: map channels to spaces and decide the migration path

1. Build `GWCHAT Mapping - Channels to Spaces - Program`. For **each in-scope team/standard channel**,

record:

• the **Google Chat space** name (name spaces consistently — e.g. one space per channel, prefixed by

the team, so the mapping is legible and reusable),

• the **member source** (the team/channel roster that seeds the space),

• the **Drive location** that holds the channel's files (the Drive workstream moves the content; the

mapping records which space points at which Drive/shared-drive location),

• the **history decision** — conversations imported, or fresh start — recorded per space.

1. Decide the **migration path** and write it into the mapping, with the limits acknowledged in writing:

• **Native channel-message migration** (channels → spaces) — supports channel *conversations* into

spaces but **does not** carry 1:1/private chats, drops some formats/features (underlining, message cards, nested lists, Streams, praise, approvals), imports into hidden **import-mode** spaces on a **90-day** clock, and caps at **1,000 teams per import** ([Google Workspace Help — run a chat migration](https://knowledge.workspace.google.com/admin/migrate/run-a-chat-migration), [Google for Developers — import data to Google Chat](https://developers.google.com/workspace/chat/import-data)).

• **Third-party tool** — where it covers more (e.g. DMs) — internals deferred to the vendor's docs

([Google Workspace Updates — migrate with CloudM](https://workspaceupdates.googleblog.com/2025/02/migrate-data-from-microsoft-teams-to-google-chat.html)).

• **Fresh start** — create spaces and members, migrate no history; the fastest, lowest-risk path when

history has little going-forward value. Author this **tool-agnostically** — the mapping governs *what maps to what and how much history*, not a specific vendor's console steps.

1. Record the **Teams source retention** decision (keep read-only / eDiscovery export) — the migration is

a productivity move, **not** a compliance archive ([Google Workspace Help — set up a chat migration](https://knowledge.workspace.google.com/admin/migrate/set-up-a-chat-migration)).

1. Freeze the mapping before Wave 0. Every wave creates spaces the same way.

Step 3 — Design and stand up meeting coexistence

1. Design `GWCHAT Meetings - Teams to Meet - Transition`:

• **Meetings → Meet.** Define how scheduled and recurring **Teams meetings** are re-issued as

Google Meet events for a wave's migrated users (new meetings created on Google Calendar/Meet; recurring Teams series re-issued as Meet series for the wave).

• **Interop during the split.** Define how cross-platform meetings work while the estate is split:

calendar sync between Google Workspace and Teams so events/RSVPs stay consistent, and **Meet↔Teams join** so each side can join the other's meetings ([Microsoft Learn — calendar syncing with Google Workspace](https://learn.microsoft.com/en-us/microsoftteams/setup-calendar-syncing-with-google-workspace), [Google Workspace Updates — Meet–Teams interoperability](https://workspaceupdates.googleblog.com/2026/02/meet-hardware-microsoft-teams-intero.html)).

1. Stand up interop on a test population and prove it: a migrated user and a not-yet-migrated user can

see each other's availability and join a single cross-platform meeting.

1. Set the interop's intended **end** — it is transitional, decommissioned as waves complete.

Step 4 — Build the wave plan

1. Segment the population into `GWCHAT Wave Plan - Segmentation - Teams` from the discovery baseline.

2. Apply the sizing rules:

• **Wave 0 = pilot** — 50–150 users of core IT and early adopters that exercises the full runbook

(spaces created, sample history imported, meetings on Meet, interop proven) before any production business unit moves.

• **Keep teams intact.** Put a whole team (and its channels' members) in the same wave so a team is

not split across Chat and Teams mid-conversation — live chat does **not** bridge 1:1 across the two platforms during coexistence.

• **Coherent business units.** Keep each wave a single business unit/geography so comms, training, and

support are focused.

• **One wave per user, fixed.** Assign every user to exactly one wave and do not move users between

waves once confirmed.

1. Build each wave's target group with `GWCHAT-Set-ScopeTargeting.ps1` (runs with `-WhatIf` by default).

Step 5 — Author the per-wave cutover runbook and go/no-go gate

1. Author `GWCHAT Runbook - Cutover - Per Wave` with the fixed five-stage sequence:

2. **Readiness** — the wave's users/groups are provisioned in Google, Chat/Meet enabled and licensed,

the wave's spaces are mapped, the Drive pre-stage for the wave's channels is complete, and comms/training are delivered; `GWCHAT-Validation.ps1` reports the wave ready.

1. **Spaces & history** — create the wave's Chat spaces from the mapping (or run the migration tool into **import-mode** spaces, then finalize within the 90-day window), seed membership, and confirm the accepted history scope. Finalizing an import-mode space is one-way — do it only when the space's history is complete for the wave.
1. **Meetings / interop** — re-issue the wave's recurring meetings as Meet; confirm calendar sync and Meet↔Teams interop are live for the wave so cross-platform meetings still work.
1. **Validation** — run the per-wave validation gates (Testing Guide) and `GWCHAT-Report.ps1` to confirm coverage (spaces created, members present, meetings working, users active).
1. **Rollback** — the documented back-out (users continue on Teams for chat/meetings; unfinalized import-mode spaces are discarded; interop keeps them working) used only if the gate fails.
1. Define `GWCHAT Gate - Go/No-Go - Per Wave`: the wave passes only when readiness held, spaces exist with correct membership, the accepted history scope imported, meetings work on Meet with interop live, validation passed, and hypercare owners are ready. The next wave does not start until this gate passes.

Step 6 — Run Wave 0 (the pilot) end to end

1. Build the Wave 0 group with `GWCHAT-Set-ScopeTargeting.ps1 -WhatIf:$false`.
2. Execute the full runbook on Wave 0: readiness → spaces & history → meetings/interop → validation.
3. Run `GWCHAT-Validation.ps1` for the wave and `GWCHAT-Report.ps1` for coverage/adoption.
4. Hold the Wave 0 `GWCHAT Gate - Go/No-Go - Per Wave` at the program cadence. The pilot passes only when the whole runbook ran end to end, the accepted history scope imported cleanly, meetings work on Meet, and interop is proven.

Step 7 — Validate the pilot and fix the runbook, not just the wave

1. Complete the Testing Guide for Wave 0. Treat every issue as a **runbook** defect — fix the runbook, the mapping, the meetings design, or the tooling choice so later waves inherit the fix.
1. Remediate any space/membership/data-quality gaps with `GWCHAT-Remediation.ps1` (runs with `-WhatIf` by default) before re-running validation.

Step 8 — Expand wave by wave

1. For each subsequent wave, re-run the same runbook: build the wave group (`GWCHAT-Set-ScopeTargeting.ps1`), execute readiness → spaces & history → meetings/interop → validation, run `GWCHAT-Validation.ps1` and `GWCHAT-Report.ps1`, and pass the go/no-go gate

before the next wave.

1. Keep interop current as the migrated population grows; each user's meetings must work on their current

platform through the split.

1. Keep adoption running per wave — usage of spaces and Meet is the real measure that a wave landed.

Step 9 — Hypercare and decommission

1. After each wave, run a heightened-support hypercare window: triage issues fast, confirm the wave's

users are active in Chat and Meet, and formally close the wave at the cadence.

1. After the **final** wave clears hypercare, decommission the meeting interop and stand down the Teams

source footprint per the design's end-state — **retaining** the Teams source (or its eDiscovery export) for as long as legal-hold/retention policy requires, never before.

Order of operations

```
Assess/baseline -> Map channels->spaces + decide path -> Meeting coexistence
-> Wave plan -> Runbook+gate -> Wave 0 pilot -> Validate/fix runbook -> Expand
wave by wave -> Hypercare + decommission (source retained for legal hold)
```

5. Testing Guide — Microsoft Teams → Google Chat & Meet Migration & Coexistence

Run these per-wave validation gates for **every** wave — first for Wave 0 (the pilot), then for each expansion wave — before that wave's go/no-go decision. Each test confirms one part of the cutover runbook worked for the wave's population. A wave does not pass until its gate criteria are met.

Test 1 — Baseline and wave assignment are complete

Do: Confirm every user in the wave is assigned to exactly one wave in `GWCHAT Wave Plan - Segmentation - Teams` and appears in the wave's target group, and that the wave keeps its teams intact; run `GWCHAT-Set-ScopeTargeting.ps1` (preview) to confirm membership. Expected: The wave's members match the plan, no user is in two waves or unassigned, and no team is split across two waves.

Test 2 — Readiness: identity, licensing, and Chat/Meet enabled

Do: Run `GWCHAT-Validation.ps1` for the wave. Expected: The wave's users/groups are provisioned into Google, licensed, and **Chat and Meet are enabled**; readiness signals report present with no gaps.

Test 3 — Spaces created with correct membership

Do: For a sample of the wave's channels, confirm the mapped **Google Chat space** exists with the name from `GWCHAT Mapping - Channels to Spaces - Program` and the correct member roster. Expected: Each sampled channel has its space, correctly named, with membership matching the mapping.

Test 4 — History scope imported as designed (and limits honored)

Do: For spaces where history was in scope, confirm channel conversations imported and the import-mode space was **finalized** within the window; confirm the accepted limits held (no 1:1/private chats expected; some formats not carried). Expected: The imported history matches the agreed scope; no space is stuck in import mode past the **90-day** deadline; unsupported content is absent as expected, not treated as a defect ([Google for Developers — import data to Google Chat](https://developers.google.com/workspace/chat/import-data), [Google Workspace Help — run a chat migration](https://knowledge.workspace.google.com/admin/migrate/run-a-chat-migration)).

Test 5 — Files reachable from the space

Do: For a sample space, confirm the channel's files are in the mapped **Google Drive / shared-drive** location and reachable by the space's members. Expected: Members can open the channel's content in Drive; the space points at the correct Drive location per the mapping (file movement itself is the Drive workstream).

Test 6 — Meetings work on Google Meet

Do: For a sample of the wave's recurring and new meetings, confirm they exist as **Google Meet** events and join successfully. Expected: Meetings resolve on Meet for migrated users; re-issued recurring series appear correctly per `GWCHAT Meetings - Teams to Meet - Transition`.

Test 7 — Coexistence: cross-platform meetings interop

Do: Create a cross-platform meeting between a migrated user (Meet) and a not-yet-migrated user (Teams); confirm calendar sync and Meet↔Teams join. Expected: Both users see the event and can join a single meeting across platforms; free/busy stays consistent ([Microsoft Learn — calendar syncing with Google Workspace](https://learn.microsoft.com/en-us/microsoftteams/setup-calendar-syncing-with-google-workspace), [Google Workspace Updates — Meet–Teams interoperability](https://workspaceupdates.googleblog.com/2026/02/meet-hardware-microsoft-teams-intero.html)).

Test 8 — Rollback is proven available

Do: Review the wave's rollback step and confirm the back-out (users continue on Teams; unfinalized import-mode spaces discarded; interop keeps them working) is documented and executable. Expected: A clear rollback exists for the wave; it is invoked only if the gate fails.

Test 9 — Adoption and coverage reported

Do: Run `GWCHAT-Report.ps1` for the wave/program. Expected: The report shows spaces created vs mapped, members present, meetings working, and users active in Chat/Meet — coverage renders without gaps for the wave.

Pass/fail gate

Gate	Requirement
Wave 0 (pilot) pass	Tests 1-9 pass; the full runbook ran end to end and every runbook defect is fixed in the runbook/mapping/design before expanding
Expansion wave pass	Tests 1-9 pass for the wave; the go/no-go gate `GWCHAT Gate - Go/No-Go - Per Wave` is cleared before the next wave starts
Program decommission	All waves passed and cleared hypercare, and the Teams source is retained per legal-hold policy, before interop and the source footprint are retired

6. Troubleshooting Guide — Microsoft Teams → Google Chat & Meet Migration & Coexistence

Use `GWCHAT-Discovery.ps1` for the Teams source baseline, `GWCHAT-Validation.ps1` for per-wave/per-team readiness, `GWCHAT-Report.ps1` for coverage/adoption, and `GWCHAT-Remediation.ps1` (runs with `-WhatIf` by default) for safe fixes to wave/space readiness data gaps. The internal errors of a specific migration tool (a single message, attachment, or space import failure) are diagnosed in that tool's own docs; this guide covers the **migration**: mapping, spaces, meetings, coexistence, and gates.

A. Users expected migrated chat history that isn't there

Symptom	Cause	Remediation
"My 1:1 chats are gone"	The native channel migration does **not** carry 1:1/private chats	Set the expectation in the adoption comms; if DMs are truly required, choose a third-party tool in the mapping — but the native path never covers them ([Google — import data to Google Chat](https://developers.google.com/workspace/chat/import-data))
"Formatting/links look wrong"	Underlining, message cards, nested lists, Streams/praise/approvals aren't migrated	Expected — not a defect. Confirm it's on the accepted-limits list in `GWCHAT Mapping - Channels to Spaces - Program` ([Google — run a chat migration](https://knowledge.workspace.google.com/admin/migrate/run-a-chat-migration))
"Old messages needed for a legal case are missing"	The migration is a productivity feature, not a compliance archive	Retrieve from the retained Teams source / eDiscovery export; never treat the migrated copy as the archive of record ([Google — set up a chat migration](https://knowledge.workspace.google.com/admin/migrate/set-up-a-chat-migration))

B. A space is stuck in import mode or was deleted

• An import-mode space is hidden from users and must be **finalized within 90 days** or it is deleted and unrecoverable. If a space vanished, it likely passed the 90-day window unfinalized — re-run the import for that channel and finalize promptly. Never open an import you can't finish; finalizing is one-way ([Google — import data to Google Chat](https://developers.google.com/workspace/chat/import-data)).

C. The import fails partway or won't take all teams

Symptom	Cause	Remediation
Import rejects teams beyond a limit	The native tool imports up to **1,000 teams per import**	Split into additional imports; plan the batches in the wave plan rather than discovering the cap mid-cutover ([Google — run a chat migration](https://knowledge.workspace.google.com/admin/migrate/run-a-chat-migration))
Messages won't import for some users	Target users aren't licensed or Chat-enabled	Enable Chat and assign licenses for the wave's users; re-run `GWCHAT-Validation.ps1` (Test 2)

D. A space has the wrong or missing members

• The space's roster doesn't match the channel. The mapping's member source is stale or the users weren't

provisioned. Re-provision the wave's users (GCDS/SCIM), reconcile the roster against `GWCHAT Mapping - Channels to Spaces - Program`, and run `GWCHAT-Remediation.ps1` (preview) then `GWCHAT-Validation.ps1`.

E. Cross-platform meetings fail during coexistence

Symptom	Cause	Remediation
Migrated and Teams users can't see each other's availability	Calendar sync not live for both directions	Confirm `GWCHAT Meetings - Teams to Meet - Transition` calendar sync is active both ways; re-run Testing Guide Test 7 ([Microsoft Learn — calendar syncing with Google Workspace](https://learn.microsoft.com/en-us/microsoftteams/setup-calendar-syncing-with-google-workspace))

Symptom	Cause	Remediation
A user can't join the other platform's meeting	Meet↔Teams interop not enabled for the wave	Confirm interop is on for the wave's users ([Google Workspace Updates — Meet–Teams interoperability](https://workspaceupdates.googleblog.com/2026/02/meet-hardware-microsoft-teams-intero.html))
A recurring meeting disappeared at cutover	Recurring Teams series was not re-issued as a Meet series	Re-issue the series on Meet per the meetings-transition design; add recurring-series re-issue to the runbook's readiness so later waves don't repeat it

F. A team is split across Chat and Teams mid-conversation

- Users report chat "stops working" between colleagues. The team was split across two waves, and live chat does not bridge 1:1 across the platforms during coexistence. Keep teams intact in one wave; if already split, accelerate the trailing members' wave to reunite the team, and confirm with `GWCHAT-Set-ScopeTargeting.ps1` (preview).

G. A wave fails its go/no-go gate

Symptom	Cause	Remediation
Readiness fails	Users not provisioned/licensed or Chat/Meet not enabled	Fix identity/provisioning and enable Chat/Meet for the wave; re-run `GWCHAT-Validation.ps1`
Validation incomplete	Spaces/history/meetings not finished for the wave	Do not pass the gate; complete the outstanding step, then re-run `GWCHAT-Report.ps1`
Gate ambiguous	Criteria not defined/owned	Complete `GWCHAT Gate - Go/No-Go - Per Wave` criteria and name the Accountable owner before deciding

H. The same problem recurs every wave

- A defect fixed only for one wave will reappear. Fix the **runbook**, the mapping, or the meetings design — not just the wave — so later waves inherit the fix (Implementation Step 7). Recurring issues are almost always an un-fixed runbook, not bad luck.

I. A cutover needs to be reversed

- The wave failed validation. Invoke the runbook's rollback: the wave's users continue on Teams for chat

and meetings, unfinalized import-mode spaces are discarded, and interop keeps them working. Rollback is only used when the gate fails — confirm it was tested (Testing Guide Test 8).

J. Graph scripts fail to connect

- `GWCHAT-Discovery.ps1`, `GWCHAT-Validation.ps1`, `GWCHAT-Report.ps1`, and

`GWCHAT-Set-ScopeTargeting.ps1` use Microsoft Graph against the source Microsoft Teams / Microsoft 365 tenant. If connection fails, confirm the Microsoft Graph PowerShell SDK is installed and the signed-in account holds the read scopes the script requests (consent on first run). Re-run after consent.

K. Diagnostics quick reference

```
.\GWCHAT-Discovery.ps1                                          # Teams estate baseline for mapping/waves
.\GWCHAT-Set-ScopeTargeting.ps1 -GroupName "..." -MembersCsv .\wave.csv     # -WhatIf by default
.\GWCHAT-Validation.ps1 -Wave "Wave 1" -MembersCsv .\wave.csv               # per-wave readiness
.\GWCHAT-Report.ps1 -MappingCsv .\gwchat-mapping.csv                        # coverage/adoption
.\GWCHAT-Remediation.ps1 -MembersCsv .\wave.csv                             # -WhatIf by default
```

L. When to escalate

If a wave's users are provisioned, licensed, and Chat/Meet-enabled, the mapping is correct, and validation still reports gaps after remediation, capture the `GWCHAT-Validation.ps1` and `GWCHAT-Report.ps1` output and review the wave with the identity, migration-tooling, and meetings owners at the program cadence before passing the go/no-go gate.

7. Best Practices — Microsoft Teams → Google Chat & Meet Migration & Coexistence

Field-proven practices for moving collaboration from Teams to Chat and Meet **without stranding users** — and without overpromising a history copy the tools can't deliver.

- **Set the honest scope on day one.** Tell users up front what moves and what doesn't: channel

conversations can migrate to spaces, but 1:1/private chats don't (native path), some formats/features don't carry, and the Teams source stays for reference and legal hold. A migration that promises a perfect history copy loses trust the moment a user notices a missing message ([Google Workspace Help — run a chat migration](https://knowledge.workspace.google.com/admin/migrate/run-a-chat-migration)).

- **Design going-forward collaboration, not a museum.** Put the effort into the *right* spaces, the

right members, meetings on Meet, and adoption — not into a byte-perfect history copy. History is "nice to have"; a working platform people actually use is the goal.

- **Map channels to spaces once, reuse every wave.** Freeze `GWCHAT Mapping - Channels to Spaces - Program`

with consistent space naming (e.g. team-prefixed, one space per channel), the member source, and the Drive location per space. Deciding naming and membership per channel at cutover is the biggest source of inconsistent spaces.

- **Keep teams intact within a wave.** Put a whole team in one wave. Live chat does **not** bridge 1:1

across Chat and Teams during coexistence, so a team split across the two platforms mid-conversation is the fastest way to strand people.

- **Identity before spaces, always.** Provision each wave's users/groups into Google (GCDS/SCIM) and

enable Chat/Meet before any space or meeting is created; a space or invite addressed to a non-existent account fails.

- **Respect the import-mode clock.** When you migrate history, spaces sit in **hidden import mode** and

must be finalized within **90 days** or they're deleted and unrecoverable; finalizing is one-way. Never open import until you can finish it, and never finalize until the wave's history is complete ([Google for Developers — import data to Google Chat](https://developers.google.com/workspace/chat/import-data)).

- **Batch within the per-import cap.** The native tool imports up to **1,000 teams per import**; plan

additional imports for larger estates rather than discovering the cap mid-cutover ([Google Workspace Help — run a chat migration](https://knowledge.workspace.google.com/admin/migrate/run-a-chat-migration)).

- **Stand up meeting interop before the first wave, and time-box it.** Calendar sync and Meet↔Teams join

must be live before anyone cuts over so migrated and not-yet-migrated users still meet — and interop is transitional, decommissioned as waves complete, never left running indefinitely ([Microsoft Learn — calendar syncing with Google Workspace](https://learn.microsoft.com/en-us/microsoftteams/setup-calendar-syncing-with-google-workspace), [Google Workspace Updates — Meet–Teams interoperability](https://workspaceupdates.googleblog.com/2026/02/meet-hardware-microsoft-teams-intero.html)).

- **Re-issue recurring meetings, don't hope they survive.** Recurring Teams series break at cutover;

re-issue them as Meet series per wave so no standing meeting is silently lost.

- **Keep the Teams source for legal hold.** The chat migration is a productivity feature, **not** a

compliance archive. Preserve the Teams source (or an eDiscovery export) for as long as retention policy requires before any decommission ([Google Workspace Help — set up a chat migration](https://knowledge.workspace.google.com/admin/migrate/set-up-a-chat-migration)).

- **Prove the whole runbook on a pilot wave.** Wave 0 (core IT + early adopters) exercises spaces,

history, meetings, and interop end to end. It's cheaper to find a defect in 100 users than in a business unit of thousands.

- **Fix the runbook, not just the wave.** Every issue found in a wave is a defect in the runbook,

mapping, or meetings design — fix it there so all later waves inherit the fix.

- **Adoption runs in tandem with the technical plan.** Comms and role-based training (what moved, where

files are, how meetings changed) are part of each wave's readiness from the start, not bolted on at cutover — usage of spaces and Meet is what tells you a wave actually landed.

• **Gate every cutover.** No wave starts until the previous wave passes `GWCHAT Gate - Go/No-Go - Per Wave`;

a wave declared done before its validation completes is an incident waiting to surface.

• **Always have a rollback.** Users can stay on Teams for chat/meetings and unfinalized import-mode spaces can be discarded — interop keeps the split estate working, so a failed wave is recoverable.

• **Keep the migration tool-agnostic.** This solution governs *what maps to what, how much history, and when*; the specific tool's console steps and the file movement live in the tool's docs and the Drive migration solution — so a tooling change touches that layer, not the whole program.

8. Deliverables Summary — Microsoft Teams → Google Chat & Meet Migration & Coexistence

Documentation

Document	Purpose
Executive Summary	Business case and what's in the box
1. Solution Overview	The Teams → Chat/Meet model (channels→spaces, meetings→Meet, files, coexistence), the honest message-history scope, and the named artifacts (the why)
2. Requirements Guide	Every prerequisite with Verify / Implement-if-missing
Global Requirements (Google)	Category-wide prerequisites, published once
3. Implementation Guide	Exact, ordered steps: baseline, map channels→spaces, decide the path, meeting coexistence, wave plan, per-wave cutover runbook, go/no-go gates, hypercare
5. Testing Guide	Per-wave validation gates with pass/fail criteria (spaces created, meetings work, users active)
6. Troubleshooting Guide	Field issues and remediation
7. Best Practices	Adoption and coexistence without stranding users

Named artifacts

Artifact	Name
Source Teams estate baseline	`GWCHAT Discovery - Teams Estate - Baseline`
Channel → space mapping	`GWCHAT Mapping - Channels to Spaces - Program`
Meetings transition design	`GWCHAT Meetings - Teams to Meet - Transition`

Artifact	Name
Coexistence design	`GWCHAT Coexistence - Chat & Meet - Transition`
Wave plan	`GWCHAT Wave Plan - Segmentation - Teams`
Per-wave cutover runbook	`GWCHAT Runbook - Cutover - Per Wave`
Go/no-go gate	`GWCHAT Gate - Go/No-Go - Per Wave`
Adoption & comms plan	`GWCHAT Adoption - Change & Comms - Program`

PowerShell scripts

Script	What it does
`GWCHAT-Discovery.ps1`	Baselines the Microsoft Teams source estate for the migration: teams, standard channels per team, channel membership, and group signals (Microsoft Graph, read-only)
`GWCHAT-Validation.ps1`	Confirms per-wave/per-team migration & coexistence readiness: the wave's users are provisioned, licensed, and Chat/Meet-enabled, and mapping/interop signals are asserted (read-only)
`GWCHAT-Report.ps1`	Renders coverage/adoption: spaces created vs channels mapped, users active, and per-wave coverage (read-only)
`GWCHAT-Remediation.ps1`	Safe remediation of wave/space readiness data gaps in a members/mapping CSV; runs with `-WhatIf` by default
`GWCHAT-Set-ScopeTargeting.ps1`	Builds a wave's target group in Microsoft Entra from a members CSV; runs with `-WhatIf` by default

The Migration playbook (outline)

A single reusable playbook the migration runs from, assembled from the artifacts above:

1. **Charter** — scope (teams/channels/meetings in scope), the honest history scope, the operating

model's stages, and the governance owners.

1. **Discovery baseline** — `GWCHAT Discovery - Teams Estate - Baseline`: teams, channels, membership,

meeting/file signals, and integrations.

1. **Channel** → **space mapping** — `GWCHAT Mapping - Channels to Spaces - Program`: space name, member

source, Drive location, and history decision per channel, plus the chosen migration path.

1. **Meetings transition** — `GWCHAT Meetings - Teams to Meet - Transition`: re-issue recurring meetings

on Meet and the interop (calendar sync + Meet↔Teams) with its decommission point.

1. **Wave plan** — `GWCHAT Wave Plan - Segmentation - Teams`: Wave 0 pilot, teams kept intact, coherent

business-unit waves, one user per wave.

1. **Per-wave cutover runbook** — `GWCHAT Runbook - Cutover - Per Wave`: readiness → spaces & history →

meetings/interop → validation → rollback.

1. **Go/no-go gate** — `GWCHAT Gate - Go/No-Go - Per Wave`: the criteria each wave clears before the

next starts.

1. **Adoption, hypercare & decommission** — `GWCHAT Adoption - Change & Comms - Program`, per-wave

hypercare, and end-of-program interop decommission with the Teams source retained for legal hold.

What the buyer receives

- This implementation guide as a Word document and PDF.
- All five PowerShell scripts in a single ZIP.
- A repeatable Teams → Chat/Meet migration following the model: Assess, then Configure, then Pilot, then

Validate, then Expand.

Outcome

Collaboration moved from Teams to Chat and Meet — channels mapped to spaces, meetings transitioned to Meet, users kept active through coexistence, and the honest history scope set and met — proven on a pilot wave before scaling, with the Teams source retained for legal hold until the estate is fully migrated.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

GWCHAT-Discovery.ps1

Purpose: Captures the Teams source-estate baseline the channel-to-space mapping and wave plan are built from: the count of teams, their standard channels, per-channel membership, and group signals, read from the Microsoft Teams / Microsoft 365 source through Microsoft Graph. Read-only — it supports the Assess step and moves no messages, creates no spaces, and changes nothing on either platform. File volumes, meeting counts, and integrations the API cannot read are recorded by hand into the baseline artifact. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\GWCHAT-Discovery.ps1 Optional: -CsvPath .\gwchat-baseline.csv to export a per-team channel summary. Optional: -MaxTeams 200 to sample large estates. Requires the Microsoft Graph PowerShell SDK. On first run, consent to the read scopes: Team.ReadBasic.All, Channel.ReadBasic.All, ChannelMember.Read.All, Group.Read.All. EXPECTED RESULTS A baseline summary: total teams, total standard channels, average channels per team, and a per-team channel/member table, with a PASS/GAP line per signal you record against the Requirements checklist as GWCHAT Discovery - Teams Estate - Baseline. .NOTES White-label — the repeatable way. Read-only against Microsoft Graph. #> [CmdletBinding()] param([string]$CsvPath, [int]$MaxTeams = 0)

How to run: powershell -ExecutionPolicy Bypass -File .\GWCHAT-Discovery.ps1 Optional: -CsvPath .\gwchat-baseline.csv to export a per-team channel summary. Optional: -MaxTeams 200 to sample large estates. Requires the Microsoft Graph PowerShell SDK. On first run, consent to the read scopes: Team.ReadBasic.All, Channel.ReadBasic.All, ChannelMember.Read.All, Group.Read.All. EXPECTED RESULTS A baseline summary: total teams, total standard channels, average channels per team, and a per-team channel/member table, with a PASS/GAP line per signal you record against the Requirements checklist as GWCHAT Discovery - Teams Estate - Baseline. .NOTES White-label — the repeatable way. Read-only against Microsoft Graph. #> [CmdletBinding()] param([string]$CsvPath, [int]$MaxTeams = 0)

GWCHAT-Remediation.ps1

Purpose: Cleans common data-quality gaps that block a wave from passing readiness, working only on a local CSV — never on tenant data, spaces, meetings, or money. It flags duplicate UPNs, blank UPN rows, and (when a source is reachable) UPNs that do not resolve in the Microsoft Teams / Microsoft 365 source, and when applied writes a normalized, de-duplicated CSV you can re-use. Runs with -WhatIf by default, so it only reports unless you explicitly turn -WhatIf off. HOW TO RUN Preview (default): powershell -ExecutionPolicy Bypass -File .\GWCHAT-Remediation.ps1 `-MembersCsv .\wave1-users.csv` `Apply: ...\GWCHAT-Remediation.ps1 -MembersCsv .\wave1-users.csv` -OutputPath .\wave1-users.clean.csv -Resolve -WhatIf:$false -Resolve additionally checks each UPN against the source tenant (needs the Microsoft Graph SDK and User.Read.All). Without -Resolve, only local de-duplication and blank checks run. The CSV is expected to have a 'Upn' column (a wave members list or a mapping export keyed by member UPN). EXPECTED RESULTS In preview: a list of duplicate/blank (and, with -Resolve, unresolved) UPNs and what would change. When applied: a cleaned CSV at -OutputPath (duplicates collapsed, blanks dropped), leaving the original untouched. .NOTES White-label — the repeatable way. Local-file only unless -Resolve is used; -WhatIf by default. #> [CmdletBinding(SupportsShouldProcess = $true, ConfirmImpact = 'Medium')] param([Parameter(Mandatory)][string]$MembersCsv, [string]$OutputPath, [switch]$Resolve)

How to run: Preview (default): powershell -ExecutionPolicy Bypass -File .\GWCHAT-Remediation.ps1 `-MembersCsv .\wave1-users.csv Apply: ...\GWCHAT-Remediation.ps1 -MembersCsv .\wave1-users.csv` -OutputPath .\wave1-users.clean.csv -Resolve -WhatIf:$false -Resolve additionally checks each UPN against the source tenant (needs the Microsoft Graph SDK and User.Read.All). Without -Resolve, only local de-duplication and blank checks run. The CSV is expected to have a 'Upn' column (a wave members list or a mapping export keyed by member UPN). EXPECTED RESULTS In preview: a list of duplicate/blank (and, with -Resolve, unresolved) UPNs and what would change. When applied: a cleaned CSV at -OutputPath (duplicates collapsed, blanks dropped), leaving the original untouched. .NOTES White-label — the repeatable way. Local-file only unless -Resolve is used; -WhatIf by default. #> [CmdletBinding(SupportsShouldProcess = $true, ConfirmImpact = 'Medium')] param([Parameter(Mandatory)][string]$MembersCsv, [string]$OutputPath, [switch]$Resolve)

GWCHAT-Report.ps1

Purpose: Produces the standing coverage/adoption report: channels mapped vs spaces created, users planned vs active, and per-wave coverage. It reads the channel-to-space mapping from a CSV (one row per channel with its target space and status) and optionally cross-checks the total source population against the Microsoft Teams / Microsoft 365 source through Microsoft Graph for the coverage denominator. Read-only against Graph — it modifies nothing. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\GWCHAT-Report.ps1 ` -MappingCsv .\gwchat-mapping.csv -CsvPath .\gwchat-coverage.csv The mapping CSV should include columns: Team, Channel, Space, Wave, SpaceCreated (true/false), MembersActive (integer), MembersPlanned (integer), Status (Planned/InProgress/Complete). Requires the Microsoft Graph PowerShell SDK and User.Read.All (used only to read the source-user count for the coverage denominator; skipped if Graph is unavailable). EXPECTED RESULTS A per-wave coverage table (channels, spaces created, members active vs planned), program totals (spaces created vs mapped, users active, overall coverage %), and an optional CSV export. No source data is modified. .NOTES White-label — the repeatable way. Read-only against Microsoft Graph. #> [CmdletBinding()] param([Parameter(Mandatory)][string]$MappingCsv, [string]$CsvPath)

How to run: powershell -ExecutionPolicy Bypass -File .\GWCHAT-Report.ps1 ` -MappingCsv .\gwchat-mapping.csv -CsvPath .\gwchat-coverage.csv The mapping CSV should include columns: Team, Channel, Space, Wave, SpaceCreated (true/false), MembersActive (integer), MembersPlanned (integer), Status (Planned/InProgress/Complete). Requires the Microsoft Graph PowerShell SDK and User.Read.All (used only to read the source-user count for the coverage denominator; skipped if Graph is unavailable). EXPECTED RESULTS A per-wave coverage table (channels, spaces created, members active vs planned), program totals (spaces created vs mapped, users active, overall coverage %), and an optional CSV export. No source data is modified. .NOTES White-label — the repeatable way. Read-only against Microsoft Graph. #> [CmdletBinding()] param([Parameter(Mandatory)][string]$MappingCsv, [string]$CsvPath)

GWCHAT-Set-ScopeTargeting.ps1

Purpose: Creates (or reuses) the Microsoft Entra security group that defines a Teams -> Chat/Meet migration wave — the pilot (Wave 0) first, then each expansion wave — so every wave runs against a bounded, well-understood population and each team stays intact within one wave. Optionally seeds members from a CSV of user UPNs. This is the only included script that writes to the tenant: it creates a group and adds members, nothing else — no messages migrated, no spaces or meetings created, no data moved. Runs with -WhatIf by default. HOW TO RUN Preview (default): powershell -ExecutionPolicy

Bypass -File .\GWCHAT-Set-ScopeTargeting.ps1 `-GroupName "GWCHAT Wave - Wave 1 - Finance EMEA" -MembersCsv .\wave1-users.csv Apply: ...\GWCHAT-Set-ScopeTargeting.ps1` -GroupName "GWCHAT Wave - Wave 1 - Finance EMEA" -MembersCsv .\wave1-users.csv ` -WhatIf:$false MembersCsv is a CSV with a 'Upn' column (users). Requires the Microsoft Graph PowerShell SDK and Group.ReadWrite.All (and User.Read.All to resolve UPNs). EXPECTED RESULTS In preview: the group that would be created/reused and the members that would be added. When applied: the security group exists and contains the listed members, ready to target with the wave's cutover runbook. .NOTES White-label — the repeatable way. Writes to Microsoft Entra; -WhatIf by default. #> [CmdletBinding(SupportsShouldProcess = $true, ConfirmImpact = 'High')] param([Parameter(Mandatory)][string]$GroupName, [string]$MembersCsv)

How to run: Preview (default): powershell -ExecutionPolicy Bypass -File .\GWCHAT-Set-ScopeTargeting.ps1 `-GroupName "GWCHAT Wave - Wave 1 - Finance EMEA" -MembersCsv .\wave1-users.csv Apply: ...\GWCHAT-Set-ScopeTargeting.ps1` -GroupName "GWCHAT Wave - Wave 1 - Finance EMEA" -MembersCsv .\wave1-users.csv ` -WhatIf:$false MembersCsv is a CSV with a 'Upn' column (users). Requires the Microsoft Graph PowerShell SDK and Group.ReadWrite.All (and User.Read.All to resolve UPNs). EXPECTED RESULTS In preview: the group that would be created/reused and the members that would be added. When applied: the security group exists and contains the listed members, ready to target with the wave's cutover runbook. .NOTES White-label — the repeatable way. Writes to Microsoft Entra; -WhatIf by default. #> [CmdletBinding(SupportsShouldProcess = $true, ConfirmImpact = 'High')] param([Parameter(Mandatory)][string]$GroupName, [string]$MembersCsv)

GWCHAT-Validation.ps1

Purpose: Validates that a migration wave is ready to cut over from Teams to Chat and Meet: the wave's users (from a members CSV) exist in the Microsoft Teams / Microsoft 365 source, are licensed, and the wave's channel-to-space mapping, meeting-coexistence, and adoption signals are asserted. This is the readiness gate for the per-wave cutover runbook. Read-only against Microsoft Graph — it changes nothing on either platform. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\GWCHAT-Validation.ps1 -Wave "Wave 1" ` -MembersCsv .\wave1-users.csv -MappingReady -InteropLive -AdoptionReady -MembersCsv is a CSV with a 'Upn' column listing the wave's users. -MappingReady asserts the channel->space mapping is frozen for the wave; -InteropLive asserts Meet<->Teams / calendar-sync interop is live; -AdoptionReady asserts comms/training are delivered. Requires the Microsoft Graph PowerShell SDK and User.Read.All. EXPECTED RESULTS "WAVE READY" when every listed user resolves and is licensed and the mapping, interop, and adoption signals are asserted; otherwise a per-signal GAP list and a non-zero exit code. .NOTES White-label — the repeatable way. Read-only against Microsoft Graph. #> [CmdletBinding()] param([string]$Wave = 'Wave 0', [Parameter(Mandatory)][string]$MembersCsv, [switch]$MappingReady, [switch]$InteropLive, [switch]$AdoptionReady)

How to run: powershell -ExecutionPolicy Bypass -File .\GWCHAT-Validation.ps1 -Wave "Wave 1" ` -MembersCsv .\wave1-users.csv -MappingReady -InteropLive -AdoptionReady -MembersCsv is a CSV with a 'Upn' column listing the wave's users. -MappingReady asserts the channel->space mapping is frozen for the wave; -InteropLive asserts Meet<->Teams / calendar-sync interop is live; -AdoptionReady asserts comms/training are delivered. Requires the Microsoft Graph PowerShell SDK and User.Read.All. EXPECTED RESULTS "WAVE READY" when every listed user resolves and is licensed and the mapping, interop, and adoption signals are asserted; otherwise a per-signal GAP list and a non-zero exit code. .NOTES White-label — the repeatable way. Read-only against Microsoft Graph. #> [CmdletBinding()]

```
param( [string]$Wave = 'Wave 0', [Parameter(Mandatory)][string]$MembersCsv, [switch]$MappingReady, [switch]$InteropLive, [switch]$AdoptionReady )
```